

Introduction to Machine Learning for Text Analysis with Python

Day 2 - Tuesday

»Unsupervised Machine Learning«

Rupert Kiddle
Sjoerd Stolwijk

r.t.kiddle@vu.nl, @rptkiddle
s.b.stolwijk@uu.nl

September 15, 2026

Today

Four use cases of machine learning

Finding similar variables

- An introduction to dimensionality reduction

- Principal Component Analysis and Singular Value Decomposition

Finding similar cases

- k-means clustering

- Beyond k-means

Important notes

Four use cases

Let's distinguish four use cases. . .

1. Finding similar variables (dimensionality reduction) – unsupervised
2. Finding similar cases (clustering) – unsupervised
3. Predicting a continuous variable (regression) – supervised
4. Predicting group membership (classification) – supervised

	x1	x2	x3	x4	x5	y
case1	■	■	■	■	■	■
case2	■	■	■	■	■	■
case3	■	■	■	■	■	■
case4	■	■	■	■	■	■

	x1	x2	x3	x4	x5	(y)
case1						
case2						
case3						
case4						

Dimensionality reduction: finding similar variables (features)

	x1	x2	x3	x4	x5	(y)
case1	■	■	■	■	■	(■)
case2	■	■	■	■	■	(■)
case3	■	■	■	■	■	(■)
case4	■	■	■	■	■	(■)

Clustering: finding similar cases

	x1	x2	x3	x4	x5	→	y
case1	■	■	■	■	■	→	■
case2	■	■	■	■	■	→	■
case3	■	■	■	■	■	→	■
case4	■	■	■	■	■	→	■
new case	■	■	■	■	■	→	?

Regression and classification: learn how to predict y.

Note, again, that the ■ signs can be *anything*. For us, often word counts or *tf* · *idf* scores (x) and, for supervised approaches, a topic, a sentiment, or similar (y).

But it could also be pixel colors or clicks on links or anything else.

	x1	x2	x3	x4	x5	y
case1	■	■	■	■	■	■
case2	■	■	■	■	■	■
case3	■	■	■	■	■	■
case4	■	■	■	■	■	■

A lot of applications and use cases, . . .

. . . but we'll distinguish two today:

1. Finding similar variables (dimension reduction)
2. Finding similar cases (clustering)

Are we more interested in which features “belong together” or which cases “belong together”?

A lot of applications and use cases, . . .

. . . but we'll distinguish two today:

1. Finding similar variables (dimension reduction)
2. Finding similar cases (clustering)

Are we more interested in which features “belong together” or which cases “belong together”?

Finding similar variables

Finding similar variables

An introduction to dimensionality reduction

Finding similar variables

An introduction to dimensionality reduction

Dimensionality reduction

dimensionality = the number of features we have

(1) Explorative data analysis and visualization

- No good way to visualize 10,000 dimensions (or even 4)

(2) The curse of dimensionality

More features means more data (good!), but:

- Too many features can lead to unfeasible computation times
- We need more training cases to increase the likelihood that the possible combinations actually occur

Dimensionality reduction

dimensionality = the number of features we have

(1) Explorative data analysis and visualization

- No good way to visualize 10,000 dimensions (or even 4)

(2) The curse of dimensionality

More features means more data (good!), but:

- Too many features can lead to unfeasible computation times
- We need more training cases to increase the likelihood that the possible combinations actually occur

Dimensionality reduction

First approach: feature selection

- Only choose the features that are really relevant

Example: Exclude all terms that occur in more than 50% of the documents, or in less than $n = 5$ documents:

```
1 vec = CountVectorizer(max_df=0.5, min_df=5)
```

https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.CountVectorizer.html

Dimensionality reduction

Second approach: feature extraction

- Create a smaller set of features
- E.g.: 1,000 features → PCA to reduce to 50 components → SML with these 50 component scores as features

Dimensionality reduction

So, we can use unsupervised ML as a dimension reduction step in a supervised ML pipeline. But it can also be a goal in itself, to understand the data better or to visualize them.

Finding similar variables

Principal Component Analysis and
Singular Value Decomposition

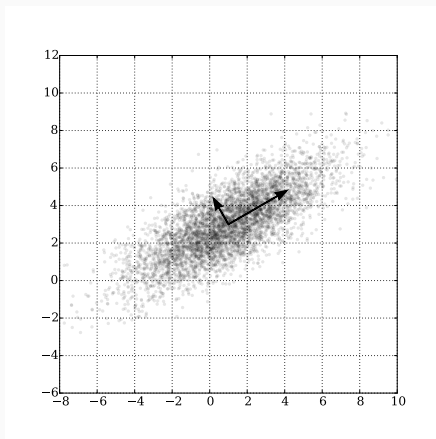
Finding similar variables

Principal Component Analysis (PCA) and Singular Value Decomposition (SVD)

PCA

- related to and often confused with Factor Analysis (same menu item in SPSS – many people who believe they run FA actually run PCA!)
- Components are ordered (first explains most variance)
- Components do *not* necessarily carry a meaningful interpretation

PCA



<https://upload.wikimedia.org/wikipedia/commons/f/f5/GaussianScatterPCA.svg>

Preparation: get some texts

As in the notebook: all 20 newsgroups, with the metadata that would give the answer away stripped out.

```
1 from sklearn.datasets import fetch_20newsgroups
2 from sklearn.decomposition import PCA
3 from sklearn.feature_extraction.text import TfidfVectorizer
4
5 newsgroups = fetch_20newsgroups(
6     subset='all',
7     remove=('headers', 'footers', 'quotes'))
8
9 # drop near-empty posts
10 docs = [d for d in newsgroups.data if len(d.strip()) > 50]
```

We do *not* use the newsgroup labels while training (hold them out for checking how well our clustering captured them...)

Running PCA

PCA does not accept a *sparse matrix* as input (but the vectorizer gives one as output), so we first convert it to a *dense matrix*.

```
1 vectorizer = TfidfVectorizer(  
2     stop_words='english', min_df=5, max_features=2000)  
3 X_tfidf = vectorizer.fit_transform(docs)  
4  
5 X_dense = X_tfidf.toarray()      # sparse -> dense  
6  
7 pca = PCA(n_components=100, random_state=42)  
8 X_pca = pca.fit_transform(X_dense) # (n_docs, 100)
```

2,000 word features → 100 components per document.

Singular value decomposition

The need to use a dense matrix is *really* a problem for large feature sets (which we have in NLP).

For larger vocabularies we can use SVD, which is essentially* the same, works on the sparse matrix directly, and is very simple to use:

```
1 from sklearn.decomposition import TruncatedSVD
2
3 svd = TruncatedSVD(n_components=100)
4 X_svd = svd.fit_transform(X_tfidf) # no .toarray() needed
```

(At our size, 2,000 features, the dense route is fine – the notebook uses PCA.)

* It's mathematically different, but SVD is even used “under the hood” by several PCA modules to solve PCA problems.

More info and background: [https:](https://towardsdatascience.com/pca-and-svd-explained-with-numpy-5d13b0d2a4d8)

[//towardsdatascience.com/pca-and-svd-explained-with-numpy-5d13b0d2a4d8](https://towardsdatascience.com/pca-and-svd-explained-with-numpy-5d13b0d2a4d8)

Singular value decomposition

The need to use a dense matrix is *really* a problem for large feature sets (which we have in NLP).

For larger vocabularies we can use SVD, which is essentially* the same, works on the sparse matrix directly, and is very simple to use:

```
1 from sklearn.decomposition import TruncatedSVD
2
3 svd = TruncatedSVD(n_components=100)
4 X_svd = svd.fit_transform(X_tfidf) # no .toarray() needed
```

(At our size, 2,000 features, the dense route is fine – the notebook uses PCA.)

* It's mathematically different, but SVD is even used “under the hood” by several PCA modules to solve PCA problems.

More info and background: [https:](https://towardsdatascience.com/pca-and-svd-explained-with-numpy-5d13b0d2a4d8)

[//towardsdatascience.com/pca-and-svd-explained-with-numpy-5d13b0d2a4d8](https://towardsdatascience.com/pca-and-svd-explained-with-numpy-5d13b0d2a4d8)

Finding similar cases

Finding similar cases

k-means clustering

Finding similar cases

k-means clustering

Grouping features vs grouping cases

Let's consider a corpus of several thousand user comments.

We could use PCA, or similar techniques like SVD or MDS, to

- figure out relationships between features
- see which features stand out
- get a first sense of what topics are in the corpus.

But:

- We do not learn anything about *which* texts (cases) belong to which topic
- We could use the component scores returned by `.fit_transform()` to then group our cases

⇒ Alternative: Choose the opposite approach and first find out which cases are most similar, *then* describe what features characterize each group of cases

Grouping features vs grouping cases

Let's consider a corpus of several thousand user comments.

We could use PCA, or similar techniques like SVD or MDS, to

- figure out relationships between features
- see which features stand out
- get a first sense of what topics are in the corpus.

But:

- We do not learn anything about *which* texts (cases) belong to which topic
- We could use the component scores returned by `.fit_transform()` to then group our cases

⇒ Alternative: Choose the opposite approach and first find out which cases are most similar, *then* describe what features characterize each group of cases

Grouping features vs grouping cases

Let's consider a corpus of several thousand user comments.

We could use PCA, or similar techniques like SVD or MDS, to

- figure out relationships between features
- see which features stand out
- get a first sense of what topics are in the corpus.

But:

- We do not learn anything about *which* texts (cases) belong to which topic
- We could use the component scores returned by `.fit_transform()` to then group our cases

⇒ Alternative: Choose the opposite approach and first find out which cases are most similar, *then* describe what features characterize each group of cases

Grouping features vs grouping cases

Let's consider a corpus of several thousand user comments.

We could use PCA, or similar techniques like SVD or MDS, to

- figure out relationships between features
- see which features stand out
- get a first sense of what topics are in the corpus.

But:

- We do not learn anything about *which* texts (cases) belong to which topic
- We could use the component scores returned by `.fit_transform()` to then group our cases

⇒ Alternative: Choose the opposite approach and first find out which cases are most similar, *then* describe what features characterize each group of cases

Grouping features vs grouping cases

Let's consider a corpus of several thousand user comments.

We could use PCA, or similar techniques like SVD or MDS, to

- figure out relationships between features
- see which features stand out
- get a first sense of what topics are in the corpus.

But:

- We do not learn anything about *which* texts (cases) belong to which topic
- We could use the component scores returned by `.fit_transform()` to then group our cases

⇒ Alternative: Choose the opposite approach and first find out which cases are most similar, *then* describe what features characterize each group of cases

k-means clustering

- Goal: group cases into k clusters
- k is set in advance
- Algorithm to determine k centroids (points in the middle of the cases that belong to it) such that the distances between the cases and their centroids are minimized
- non-deterministic: starts with a randomly chosen centroids (there are other versions)
- Cheap to compute: works even with large number of cases
- We can run PCA first to reduce the number of features if we want/need to

k-means clustering

- Goal: group cases into k clusters
- k is set in advance
- Algorithm to determine k centroids (points in the middle of the cases that belong to it) such that the distances between the cases and their centroids are minimized
- non-deterministic: starts with a randomly chosen centroids (there are other versions)
- Cheap to compute: works even with large number of cases
- We can run PCA first to reduce the number of features if we want/need to

k-means clustering

- Goal: group cases into k clusters
- k is set in advance
- Algorithm to determine k centroids (points in the middle of the cases that belong to it) such that the distances between the cases and their centroids are minimized
- non-deterministic: starts with a randomly chosen centroids (there are other versions)
- Cheap to compute: works even with large number of cases
- We can run PCA first to reduce the number of features if we want/need to

k-means clustering

- Goal: group cases into k clusters
- k is set in advance
- Algorithm to determine k centroids (points in the middle of the cases that belong to it) such that the distances between the cases and their centroids are minimized
- non-deterministic: starts with a randomly chosen centroids (there are other versions)
- Cheap to compute: works even with large number of cases
- We can run PCA first to reduce the number of features if we want/need to

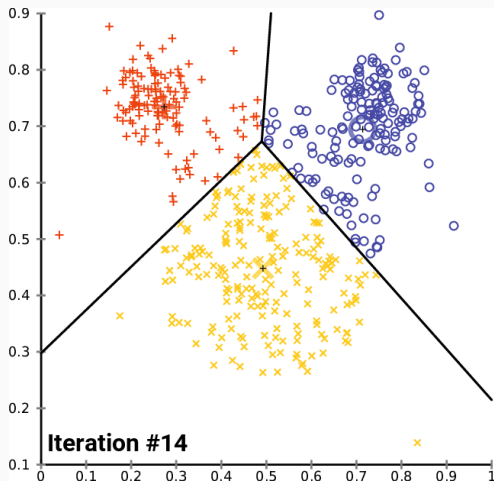
k-means clustering

- Goal: group cases into k clusters
- k is set in advance
- Algorithm to determine k centroids (points in the middle of the cases that belong to it) such that the distances between the cases and their centroids are minimized
- non-deterministic: starts with a randomly chosen centroids (there are other versions)
- Cheap to compute: works even with large number of cases
- We can run PCA first to reduce the number of features if we want/need to

k-means clustering

- Goal: group cases into k clusters
- k is set in advance
- Algorithm to determine k centroids (points in the middle of the cases that belong to it) such that the distances between the cases and their centroids are minimized
- non-deterministic: starts with a randomly chosen centroids (there are other versions)
- Cheap to compute: works even with large number of cases
- We can run PCA first to reduce the number of features if we want/need to

k-means clustering



https://upload.wikimedia.org/wikipedia/commons/e/ea/K-means_convergence.gif

Notice the big symbols indicating the centroids.

```
1 from sklearn.feature_extraction.text import TfidfVectorizer
2 from sklearn.cluster import KMeans
3
4 from sklearn.cluster import KMeans
5
6 km_raw = KMeans(
7     n_clusters=20,    # we happen to know there are 20 groups
8     n_init=10,        # 10 random restarts, keep the best
9     random_state=42)
10
11 clusters_raw = km_raw.fit_predict(X_tfidf)
```

That's it! (Here on the raw TF-IDF matrix – the reduced version comes next.)

- `clusters_raw` is a list of integers indicating the predicted cluster number. We can thus use `zip(clusters_raw, docs)` to put them together.
- We could also use `.fit()` and `.predict()` separately and use our `km_raw` to predict clusters for additional cases we have not used to train the model

```
1 from sklearn.feature_extraction.text import TfidfVectorizer
2 from sklearn.cluster import KMeans
3
4 from sklearn.cluster import KMeans
5
6 km_raw = KMeans(
7     n_clusters=20,    # we happen to know there are 20 groups
8     n_init=10,        # 10 random restarts, keep the best
9     random_state=42)
10
11 clusters_raw = km_raw.fit_predict(X_tfidf)
```

That's it! (Here on the raw TF-IDF matrix – the reduced version comes next.)

- `clusters_raw` is a list of integers indicating the predicted cluster number. We can thus use `zip(clusters_raw, docs)` to put them together.
- We could also use `.fit()` and `.predict()` separately and use our `km_raw` to predict clusters for additional cases we have not used to train the model

Using k-means clustering. . .

- we get the cluster membership for each text; and
- we get the terms that are most characteristic for the documents in each cluster.

Finding the optimal k

- The only way to find k is to estimate multiple models with different k s
- No single best solution; finding a balance between error within clusters (distances from centroid) and low number of clusters.
- An elbow plot can be helpful (see example in Burscher et al, 2016)

```
1 inertias = []  
2 for k in range(2, 31, 2):  
3     km = KMeans(n_clusters=k, n_init=4, random_state=42)  
4     km.fit(X_norm)  
5     inertias.append(km.inertia_)
```

Burscher, B., Vliegthart, R., & de Vreese, C. H. (2016). Frames beyond words: Applying cluster and sentiment analysis to news coverage of the nuclear power issue. *Social Science Computer Review*, 34(5), 530-545. doi:10.1177/0894439315596385

Finding the optimal k

- The only way to find k is to estimate multiple models with different k s
- No single best solution; finding a balance between error within clusters (distances from centroid) and low number of clusters.
- An elbow plot can be helpful (see example in Burscher et al, 2016)

```
1 inertias = []
2 for k in range(2, 31, 2):
3     km = KMeans(n_clusters=k, n_init=4, random_state=42)
4     km.fit(X_norm)
5     inertias.append(km.inertia_)
```

Burscher, B., Vliegenthart, R., & de Vreese, C. H. (2016). Frames beyond words: Applying cluster and sentiment analysis to news coverage of the nuclear power issue. *Social Science Computer Review*, 34(5), 530-545. doi:10.1177/0894439315596385

Finding similar cases

Beyond k-means

Finding similar cases

Beyond k-means

Downsides of k-means clustering

k-means is fast, but has problems:

- k can only be determined by fitting multiple models and comparing them
- bad results if the wrong k is chosen
- bad results if the (real) clusters are non-spherical
- bad results if the (real) clusters are not evenly sized

Two ideas that fix these problems

Idea 1: cluster hierarchically

Start with every document as its own cluster, then repeatedly merge the two most similar clusters. This builds a full tree of groupings (a *dendrogram*) – you choose the level of granularity *after* seeing the structure, instead of committing to a k up front.

Idea 2: define clusters by density

A cluster is a region where documents sit densely packed; wherever the space thins out, the cluster ends. Two consequences: clusters can have *any shape* (not just spheres around a centroid), and a document in no dense region is labelled an *outlier* instead of being forced into the nearest cluster.

Two ideas that fix these problems

Idea 1: cluster hierarchically

Start with every document as its own cluster, then repeatedly merge the two most similar clusters. This builds a full tree of groupings (a *dendrogram*) – you choose the level of granularity *after* seeing the structure, instead of committing to a k up front.

Idea 2: define clusters by density

A cluster is a region where documents sit densely packed; wherever the space thins out, the cluster ends. Two consequences: clusters can have *any shape* (not just spheres around a centroid), and a document in no dense region is labelled an *outlier* instead of being forced into the nearest cluster.

Better solutions for high-dimensional text data

BERTopic defaults to more modern descendants:

- **UMAP** replaces PCA. PCA can only draw straight axes through the data; UMAP instead preserves each document's *neighbourhood*, so it can follow curved and clumpy structure. Cost: more settings, and randomness (fix the seed!).
- **HDBSCAN** replaces K-Means. It builds a density-based cluster hierarchy, then selects the stable clusters from it. You set a *minimum cluster size* instead of k , and outliers are allowed to stay outliers.

Important notes

Important notes for all types of clustering

Important notes

Consider the scales of measurement

Clustering is based on distances – if your features are not measured on the same scale, or if it is not meaningful to calculate a numerical distance, it won't produce meaningful results!

Consider standardizing/whitening your features!

Pay attention outliers/extreme cases

Extreme cases or outliers can have a strong influence.

Do proper pre-processing

To reduce the number of features, but also to have *meaningful* features (dimensions on which you expect high distances between the clusters).

Important notes

Consider the scales of measurement

Clustering is based on distances – if your features are not measured on the same scale, or if it is not meaningful to calculate a numerical distance, it won't produce meaningful results!

Consider standardizing/whitening your features!

Pay attention outliers/extreme cases

Extreme cases or outliers can have a strong influence.

Do proper pre-processing

To reduce the number of features, but also to have *meaningful* features (dimensions on which you expect high distances between the clusters).

Important notes

Consider the scales of measurement

Clustering is based on distances – if your features are not measured on the same scale, or if it is not meaningful to calculate a numerical distance, it won't produce meaningful results!

Consider standardizing/whitening your features!

Pay attention outliers/extreme cases

Extreme cases or outliers can have a strong influence.

Do proper pre-processing

To reduce the number of features, but also to have *meaningful* features (dimensions on which you expect high distances between the clusters).

This afternoon: topic modelling with **BERTopic** – this morning's recipe with every part upgraded.